

exp-fips203-mlkem-kem-keygen-k3

FIPS 203 Alg.19 ML-KEM-768 KEM KeyGen 实现方案

ascendc 工程 (ML-KEM-768 W4b / E19)

2026-07-26

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-kem-keygen-k3/`: 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 19**, 即 ML-KEM-768 ($k = 3$) **KEM KeyGen**。

参数锁定: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem768-parameter-card.md §3.3 D19: ek_kem=1184B, dk_kem=2400B, SIM/生产 2 launch`, Alg.16 尾在第二个 launch 内融合。**行为基线**: 活跃探针 `ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-alg19-kem-keygen-device-k3/`; PKE 主体可由本参数组 E13 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-keygen-k3/` 与 Alg.16 KEM tail 组合。**计划 AI Core 数**: Launch 1 为 AIV_ONLY blockDim=2; Launch 2 为 MIX blockDim=1 (1 AIC + 2 AIV)。**禁止**: 创建 `examples/stable/ml-kem/ml-kem-768/`; 从任何 `frozen/` 目录复制、改写或移植源码; 沿用 k4 的 `ek=1568/dk_kem=3168` 或 `pad-to-4/8` 几何。

目录

1	工程边界	2
1.1	允许的代码来源与依赖	2
1.2	禁止项	2
2	数学语义 (Alg.19 → Alg.16 → Alg.13)	2
2.1	输入输出	2
2.2	阶段公式	3
3	AscendC 实现规划	3
3.1	Launch 与 AI Core 规模	3
3.2	GM 几何与偏移	3
3.3	同步与流水	4
4	AscendC API 表	4
4.1	评估过但未采用	5
5	数学步骤覆盖率	5

目录	2
6 验证计划	6
6.1 默认验收	6
6.2 可选对照	6
7 产出物	6

1 工程边界

1.1 允许的代码来源与依赖

来源	用途
ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203- alg19-kem-keygen-device -k3/	活跃 D19 k3 行为基线；可复制后自包含适配到本 exp，最终不得保留对探针目录的编译期 include/import
examples/incubating/ml-kem/ml-kem-768/exp-fips 203-mlkem-pke-keygen-k3 /	活跃 E13 incubating PKE KeyGen；提供已自包含的 k3 Alg.13 主体
examples/incubating/ml-kem/ml-kem-1024/exp-fips 203-mlkem-kem-keygen-k 4/	活跃 k4 incubating KEM KeyGen 结构参考；仅用于 retarget 到 k3 锁定字节长与几何
library/shared/ CANN / tikicpulib / CAModel	SHAKE/Keccak、公共设备侧头文件等共享库编译、CPU 孪生与 SIM 运行环境

1.2 禁止项

- 禁止从 ascendc-tests/frozen/ 或 examples/frozen/ 拿源码、customspec 或路线。
- 禁止把 d 、 z 、 $H(ek)$ 、 $\hat{A}$ 、 $\hat{s}$ 、 ρ 等中间态作为默认 I/O 落盘；debug dump 只能由显式开关启用。
- 禁止把 k4 的字节长、polyvec8、 $\hat{A}[16]$ 或零垫 4/8 当作默认。
- 禁止为 $H(ek)$ 或 z 单独增加第三次生产 launch。

2 数学语义 ($\text{Alg.19} \rightarrow \text{Alg.16} \rightarrow \text{Alg.13}$)

2.1 输入输出

张量 / 文件	契约
input/seed_d.bin	4B little-endian uint32，由 Host 派生 FIPS 203 的 d ；域分离串为 exp-mlkem-f203-2s1e-k3:SEED_D=
input/lut_even_stacked.bin	NTT Stage2 LUT；由本目录 golden 脚本生成；不是用户可见交付输入
lut_odd_stacked.bin	

output/ek_kem.bin	1184B, 等于 PKE public key: $\text{ByteEncode}_{12}(\hat{t})$ 的 1152B 后拼接 ρ 的 32B
output/dk_kem.bin	2400B, 展开布局为 $\text{dk_pke}(1152) \parallel \text{ek_kem}(1184) \parallel H(\text{ek})(32) \parallel z(32)$

2.2 阶段公式

1. Alg.19: 从 seed_d.bin 可复现派生 d 与 z ; d 使用 exp-mlkem-f203-2s1e-k3:SEED_D=, z 使用 exp-mlkem-f203-kem-k3:SEED_Z=。
2. Alg.16: 调用 Alg.13 生成 $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}})$ 。
3. Alg.16 尾: $\text{ek}_{\text{KEM}} = \text{ek}_{\text{PKE}}$; $\text{dk}_{\text{KEM}} = \text{dk}_{\text{PKE}} \parallel \text{ek}_{\text{KEM}} \parallel H(\text{ek}_{\text{KEM}}) \parallel z$, 其中 $H = \text{SHA3-256}$ 。
4. Alg.13 主体复用 E13 / D13 k3 几何: $\hat{A}[9, 256]$ 、 $s \parallel e[6, 256]$ 、polyvec6 NTT、InnerProduct 输出 3 行、ByteEncode12 $3 \times 384\text{B}$ 。

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 prep	AIV_ONLY, blockDim=2	继承 E13/D13: $\hat{A}$ 共 9 poly 按 5+4 分给两个 AIV; $s \parallel e$ 共 6 poly 生成到真实 polyvec6 GM。选择 2 AIV 是参数卡 §3.3 D19 对 D13 prep 的锁定继承。
2 compute+tail	MIX, blockDim=1	1 AIC + 2 AIV; 先执行 Alg.13 行 16–21, polyvec6 NTT 连续 3+3, InnerProduct 输出 2+1; 随后在同一 launch 内由约定 AIV 执行 Alg.16 KEM tail, 避免第三 launch。

3.2 GM 几何与偏移

对象	锁定几何
$\hat{A}$	$[9, 256]$ int32; 9 个真实 poly, 禁止 pad 到 16
$s \parallel e$	$[6, 256]$; polyvec6, NTT 前后都按真实 6 poly 管理
NTT S0	$[12, 256]$ int8, 即 $[\text{HI}6, \text{LO}6]$; 每个 AIV 持有完整 poly 的 hi+lo, 禁止 limbsplit
Stage2 mat_c	$[48, 128]$ int32; 逻辑 $6 \times 4 \times 2$; Cube 的 M 对齐垫到 16 只是硬件对齐, 不表示假 poly

ek_kem	1184B; 等于 ek_pke
dk_kem	2400B; 偏移: dk_pke[0,1152), ek[1152,2336), H[2336,2368), z[2368,2400)

3.3 同步与流水

Prep 内部使用 UB/GM 分段搬运与必要的 PipeBarrier; 两次 launch 之间由 Host 顺序保证 GM 可见性。Compute 内部沿用 E13/D13 已验同步: AIV 写 Stage1/Stage3 GM 后以 CrossCore flag 与 AIC 交接; AIC 写完 mat_c 后 AIV 等待再做 Stage3 merge、InnerProduct 与 ByteEncode12。Alg.16 尾必须在 ByteEncode12 与 ek fuse 完成后执行; 双 AIV 写共享 GM 时须先汇合, 再由固定 AIV 计算 $H(ek)$ 、派生 z 并拼 dk_kem。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E19 独有的新 AscendC API; 下表 API 均已有索引记录, 使用约束按 library /documents/CANN-AscendC算子开发接口参考-查阅索引.md 执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核; 禁止把 CPU 打印误认为多核性能
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	GM/UB 张量契约必须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运; 长度与地址遵守 32B 对齐约束
Duplicate	Memory 矢量计算	√	uint8/int32	用于 UB 清零、pad 与 tail buffer 初始化
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接时使用; CPU 过不能删

SyncAll	硬同步	√	—	KEM tail 前双 AIV 汇合; MIX 中 AIC 已返回后按 AIV-only 汇合语义使用
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	Stage2 MMAD; 逻辑 M=12, 硬件 M 对齐垫到 16
ShiftRight	矢量算术	√	int32	limb 拆分、mask low bits 与 Barrett 相关右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助
Cast	类型转换	√ (受限)	int32/int16/half/int8	遵守 A2 Cast 链约束; 不得假设存在 int32->int8 单跳
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 与 tail 标量缺口; 不得把 Compares bit 包当逐 lane 布尔

4.1 评估过但未采用

API / 路线	不采用原因
Compares/GatherMask compact 链	Alg.19 E19 不新增 reject compact; 沿用 E13/D13 已验采样路径, 不新增比较掩码风险
高阶 Matmul<>	本路线继承手写 MMAD + 平面 mat_c 契约; 高阶 Matmul 输出布局与 MIX 交接不作为本轮变量
第三 launch KEM tail	参数卡 §3.3 D19 锁定 2 launch; 尾段必须内嵌在 compute launch 后半段

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
------	------------	----	------

$G(d 3)$ 与 XOF	shared Keccak/SHAKE + UB 流水	100% 设备	标量 Keccak 核 属本仓 shared; 非 Host oracle
SampleNTT 生成 $\hat{A}[9]$	AIV_ONLY + rej 路径 + DataCopy	100% 设备	5+4 分片; 无零垫
CBD $\eta = 2$ 生成 $s e[6]$	AIV_ONLY + CBD LUT/向量累加	100% 设备	输出按 polyvec6 真实行数
polyvec6 NTT S1-S3	MIX AIV/AIC + Mmad	100% 设备	NTT 内无 Gather
InnerProduct $P = 3$	Alg.11 向量路径 + AIV 2+1	100% 设备	非整除分片显式处理第 3 行
ByteEncode12 + $ek \rho$	ByteEncode12 向量 + 尾部标量 pack	部分向量	pack 尾部标量缺口沿用 E13/D13 已验实现
$H(ek)$ 与 z 派生	shared Keccak SHA3-256 + tail UB 拼接	100% 设备	仅 AIV0 负责最终拼接, 前置汇合保证 GM 完整

6 验证计划

6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-kem-keygen-k3
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: ek_kem.bin 为 1184B, dk_kem.bin 为 2400B; 两者均与 golden 对拍 PASS。SIM 通过后记录 Total tick 到 qa/active_sim_regress_summary.md, 并检查用例根目录无 stray core*.dump、profile_*log*.toml。

6.2 可选对照

后续可补 liboqs-768 KAT; E19 本轮门禁为 CPU + SIM_DIRECT=1 sim。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

7 产出物

- exp-fips203-mlkem-kem-keygen-k3-实现方案-customspec.tex
- exp-fips203-mlkem-kem-keygen-k3-实现方案-customspec.pdf

- 后续实现文件须与本 customspect 同目录自包含，且自研 Python/C/AscendC 写详细中文注释。